

Ejercicio 1

Predicado mayor que 100

```
Predicate<Integer> mayor100 =  
    numero -> numero > 100;  
System.out.println(mayor100.test(150));
```

Ejercicio 2

Predicado entre 100 y 300

```
Predicate<Integer> mayor100 =  
    numero -> numero >= 100;  
Predicate<Integer> menor300 =  
    numero -> numero <= 300;  
Predicate<Integer> entre100y300 =  
    mayor100.and(menor300);  
System.out.println(entre100y300.test(200));
```

Ejercicio 3

Predicado mayor a 100 o menor que 50

```
Predicate<Integer> condicion =  
    numero -> numero > 100 || numero < 50;  
System.out.println(condicion.test(25));
```

Ejercicio 4

Predicado número distinto de 100

```
Predicate<Integer> distinto100 =  
    numero -> numero != 100;  
System.out.println(distinto100.test(80));
```

Ejercicio 5

Predicado para comparar Strings

```
BiPredicate<String, String> iguales =  
    (a, b) -> a.equals(b);  
System.out.println(iguales.test("Hola", "Hola"));
```

Ejercicio 6

Lista y predicados entre 25 y 30

```
import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class Main {

    public static void main(String[] args) {

        List<Integer> numeros = Arrays.asList(
            10, 15, 20, 25, 28,
            35, 40, 50, 60, 70
        );

        Predicate<Integer> mayor25 =
            n -> n >= 25;

        Predicate<Integer> menor30 =
            n -> n <= 30;

        boolean resultado = numeros.stream()
            .anyMatch(mayor25.and(menor30));

        System.out.println(resultado);
    }
}
```

Ejercicio 7

Predicado número igual a 22

```
Predicate<Integer> igual22 =
    numero -> numero == 22;

boolean resultado = numeros.stream()
    .anyMatch(igual22);

System.out.println(resultado);
```

Ejercicio 8

Predicados para cadenas

```
import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class Main {

    public static void main(String[] args) {

        List<String> cadenas = Arrays.asList(
            "Ana",
            "Pedro",
            "Alicia",
            "Mario",
            "Lucia",
            "Pablo"
        );

        Predicate<String> empiezaA =
            texto -> texto.startsWith("A");

        Predicate<String> longitud5 =
            texto -> texto.length() == 5;

        boolean resultadoA =
            cadenas.stream().anyMatch(empiezaA);

        boolean resultado5 =
            cadenas.stream().anyMatch(longitud5);

        System.out.println(resultadoA);
        System.out.println(resultado5);
    }
}
```

Ejercicio 9

Predicado que no empiece por P

```
Predicate<String> noEmpiezaP =  
    texto -> !texto.startsWith("P");  
cadenas.stream()  
    .filter(noEmpiezaP)  
    .forEach(System.out::println);
```

Ejercicio 10

Clase PredicadoTest

```
import java.util.function.Predicate;  
  
public class PredicadoTest  
    implements Predicate<Integer> {  
    @Override  
    public boolean test(Integer numero) {  
        return numero % 2 == 0;  
    }  
}
```

Main de prueba

```
public class Main {  
    public static void main(String[] args) {  
        PredicadoTest predicado =  
            new PredicadoTest();  
        System.out.println(predicado.test(10));  
    }  
}
```

Ejercicio 11

Clase Ciudad

```
public class Ciudad {  
    private String nombre;  
    private int habitantes;  
    public Ciudad(String nombre, int habitantes) {  
        this.nombre = nombre;  
        this.habitantes = habitantes;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public int getHabitantes() {  
        return habitantes;  
    }  
    @Override  
    public String toString() {  
        return nombre + " - "  
            + habitantes + " habitantes";  
    }  
}
```

Main

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.function.Predicate;  
public class Main {  
    public static void imprimirCiudades(  
        List<Ciudad> ciudades,  
        Predicate<Ciudad> predicado) {
```

```

    for (Ciudad c : ciudades) {
        if (predicado.test(c)) {
            System.out.println(c);
        }
    }
}

public static void main(String[] args) {
    List<Ciudad> ciudades = new ArrayList<>();
    ciudades.add(new Ciudad("Elche", 230000));
    ciudades.add(new Ciudad("Elda", 52000));
    ciudades.add(new Ciudad("Madrid", 3200000));
    ciudades.add(new Ciudad("Alicante", 330000));
    ciudades.add(new Ciudad("Eibar", 27000));
    ciudades.add(new Ciudad("El Escorial", 150000));
    ciudades.add(new Ciudad("Bilbao", 340000));
    ciudades.add(new Ciudad("Eivissa", 50000));
    ciudades.add(new Ciudad("Valencia", 800000));
    ciudades.add(new Ciudad("Ecija", 120000));
    Predicate<Ciudad> filtro =
        ciudad ->
            ciudad.getNombre().startsWith("E")
            && ciudad.getHabitantes() > 100000;
    imprimirCiudades(ciudades, filtro);
}
}

```

Ejercicio 12

Comparar ciudades con Predicate

```
BiPredicate<Ciudad, Ciudad> iguales =  
    (c1, c2) ->  
        c1.getNombre().equals(c2.getNombre());  
Ciudad ciudad1 =  
    new Ciudad("Elche", 230000);  
Ciudad ciudad2 =  
    new Ciudad("Elche", 100000);  
System.out.println(  
    iguales.test(ciudad1, ciudad2));
```

Ejercicio 13

Uso de IntPredicate

Sí, podríamos haber utilizado IntPredicate en ejercicios donde trabajamos con enteros.

Ejemplo

```
IntPredicate mayor100 =  
    numero -> numero > 100;  
System.out.println(mayor100.test(150));  
IntPredicate evita conversiones innecesarias entre int e Integer.
```

Ejercicio 14

Predicado usando clase anónima

```
Predicate<Integer> mayor100 =  
    new Predicate<Integer>() {  
        @Override  
        public boolean test(Integer numero) {  
            return numero > 100;  
        }  
    };  
System.out.println(mayor100.test(150));
```

Ejercicio 15

Otra forma de usar Predicate

Sí. También podemos usar referencias a métodos.

```
import java.util.function.Predicate;

public class Main {

    public static boolean esPositivo(int n) {

        return n > 0;

    }

    public static void main(String[] args) {

        Predicate<Integer> predicado =

            Main::esPositivo;

        System.out.println(predicado.test(5));

    }

}
```